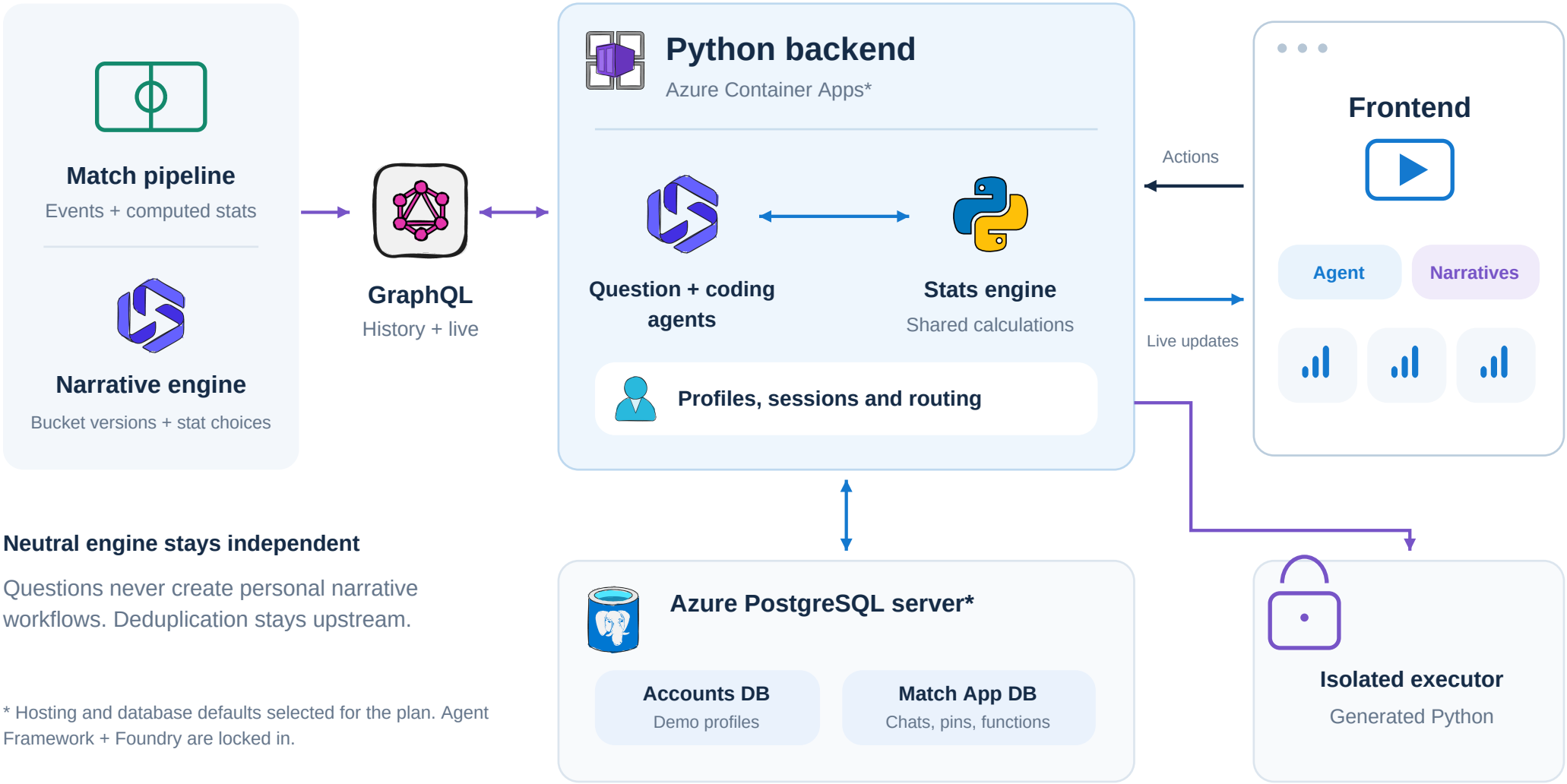
One match. One shared backend.

GraphQL brings the match in. The backend turns it into each viewer's experience.



Neutral engine stays independent

Questions never create personal narrative workflows. Deduplication stays upstream.

* Hosting and database defaults selected for the plan. Agent Framework + Foundry are locked in.

Ask once. Build, test, then pin.

The question agent defines the goal. A specialist coding agent writes the function.

VIEWER **"How many headers has Chelsea made since minute 20?"**

1 **Understand**
Meaning + available data

→

2 **Write function**
Python / regex / filters

→

3 **Check result**
Test against the goal

→

4 **Calculate**
History, then live events

→

5 **Offer to pin**
A running stat card

Up to 3 repairs, then explain failure

Controlled execution

Fixed libraries + read-only GraphQL.
Validate before use; no ongoing semantic checks.
Equivalent requests share one match-scoped function.

PINNING REPLACES THE LOWEST DEFAULT

Goals	Cards	Possession
Goals	Cards	**Headers***

* Illustrative 3-slot layout. Pinning is a separate viewer choice.

Twelve views of shared intelligence.

The narrative engine supplies the wording, target bucket and ranked default stat changes.

Demo profile

Supported team + fan style

ONE LIVE MATCH



UP TO 12 VERSIONS - ONLY RELEVANT BUCKETS

An insight may have four versions for one team, or versions for all twelve buckets. The user backend routes them without rewriting.

Joining mid-match loads the backlog and current bucket selection.

"New stats"
Bucket + priority order

→

goals, cards, ...
Approved stat pool

→

Fetch values; preserve pins

Build the live experience first.

A small implementation sequence, with the second version kept visibly separate.

PHASE 1



1. Connect

Python API, GraphQL and demo profiles



2. Answer

Agent Framework + Foundry models



3. Calculate

Executor, test harness and shared stat state



4. Deliver

Live cards, narratives and session recovery

One profile = one active tab. New tabs take over. Work survives a viewer disconnect.

PHASE 2



Semantic subscriptions

Jev checks incoming narratives. Show a matching insight linked to the request.



Highlights + replay

Return ordered sources and time ranges. The frontend plays the segments.



Follow playback time

Rewind narratives, stat values and bucket selections. Avoid future spoilers.

To finalise: sandbox technology, model deployments, live transport and API contracts.

Basis: agreed architecture, 18 Oct 2026.

Azure GraphQL Jev Icon library